

Tutorium 6

Higher Order Functions: Map, Fold & Filter

Grundlagen der Programmierung

15. Juni 2026

Lambda-Funktionen

Map & Filter

Fold

Praxis

Anonyme Funktionen (Lambdas)

Lambda-Funktionen sind anonyme Funktionen, die man “on the fly” erstellen kann, ohne ihnen einen Namen zu geben.

Syntax einer Lambda-Funktion

```
1  -- Normal :  
2  addOne' :: Int -> Int  
3  addOne' x = x + 1  
4  
5  -- Mit Lambda :  
6  addOne  :: Int -> Int  
7  addOne x = (\x -> x + 1) x
```

Startet mit \ (Backslash), gefolgt von Parametern, einem Pfeil -> und dem Ausdruck.

Die map Funktion

map wendet eine Funktion auf **jedes** Element einer Liste an. Die Länge und Reihenfolge der Liste bleibt erhalten!

Beispiel: Map

```
1  -- Alle Elemente quadrieren
2  squareListMap :: Num a => [a] -> [a]
3  squareListMap xs = map (\x -> x^2) xs
4
5  -- Nur gerade Zahlen verdoppeln
6  doubleEvens :: [Int] -> [Int]
7  doubleEvens xs = map (\x -> if even x then x * 2 else x) xs
```

Die filter Funktion

`filter` prüft jedes Element gegen eine Bedingung (Prädikat) und behält nur jene, für die `True` zurückgegeben wird.

Beispiel: Filter

```
1 filterEvens :: [Int] -> [Int]
2 filterEvens xs = filter (\x -> even x) xs
3 -- Aufruf: filterEvens [1,2,3,4,5,6] -> [2,4,6]
4
5 filterGreaterThanFive :: [Int] -> [Int]
6 filterGreaterThanFive xs = filter (\x -> x > 5) xs
```

foldr und foldl

fold reduziert eine Liste auf einen einzigen Wert (z. B. Summe).

- ▶ foldr (Right): Faltet von rechts nach links. Baut den Term rekursiv auf.
- ▶ foldl (Left): Faltet von links nach rechts.

Wichtig bei nicht-kommutativen Operationen (z.B. Division)

```
1 foldDivr xs = foldr (/) 1 xs
2 -- [2,4,8] => 2 / (4 / (8 / 1)) = 4.0
3
4 foldDivl xs = foldl (/) 1 xs
5 -- [2,4,8] => ((1 / 2) / 4) / 8 = 0.015625 (1/64)
```

Live Demo



01_map.hs, 02_fold.hs, 03_filter.hs

Wir programmieren Higher Order Functions live im Code.